

NATS clustering: how far do three nodes actually **get you?**

Three demos in this lab were built to answer one growth question, and every claim on the slides that follow is a measured number, not a design opinion.

THE QUESTION

We start on one three-instance cluster. When must we add a second cluster, a gateway, an arbiter site or a leaf node — and what does JetStream cost us at each step?

THE EVIDENCE BEHIND THESE SLIDES

■ Demo 02 — multi-region · 6 servers, gateway, accounts

■ Demo 03 — 7 topologies · 73 checks, 0 failed

■ Demo 04 — JetStream + CQRS · rehydration cost

01

WHAT WAS ACTUALLY RUN

Three demos, one growth question

Each demo holds everything still except one variable. Demo 02 varies the account model on a fixed topology. Demo 03 varies the topology itself. Demo 04 drops to a single server and measures what JetStream costs when you rebuild state from the log.

DEMO 02 — MULTI-REGION

- > **6 servers**, two clusters of three, joined by one **gateway**.
- > Variable: the **account model**. One shared account, or one per region.
- > Answers: does a tenant wall hold across a gateway? What does **Replicas: 1** cost? Where does the data physically live?

DEMO 03 — TOPOLOGY AS THE VARIABLE

- > **7 runnable shapes** needing 6, 7 or 9 servers.
- > Every shape built from nothing by a script, measured, then torn down.
- > **73 checks passed, 0 failed**, plus 19 recorded observations. A check has an expected answer and passes only on an exact match.

DEMO 04 — JETSTREAM + CQRS

- > **1 server**. No cluster, no gateway, no domain.
- > Variable: rebuild an aggregate **with** a snapshot, or **without** one.
- > Answers: what does replay cost, and how fast does that cost grow?

THE FIVE WORDS THAT KEEP GETTING CONFUSED

WORD	QUESTION IT ANSWERS	PLAIN MEANING
Region	Where does it run?	A place. Cape Town. Sydney.
Tenant	Whose is it?	A paying customer. A commercial fact, not a running thing.
Account	What may it touch?	A NATS wall. It decides what you may see and send.
Cell	What ships together?	One stamp of the stack, deployed as a unit.
JetStream domain	Which storage pool holds the log?	Which NATS storage the messages actually live in.

WHY THIS SLIDE EXISTS

These five are **independent axes**. No two may be defined as permanently equal. One V3 drawing used the word **“domain” 26 times** on one page, carrying three unrelated meanings.

Only one is enforced by a machine: **NATS enforces the account wall**. The rest are conventions we hold ourselves. Region and cell are places; tenant and account are not.

02

FOUR WORDS USED ON EVERY SLIDE THAT FOLLOWS

Instance, cluster, gateway, leaf

The shape below is the one we would actually deploy: two AWS regions, three availability zones each, one NATS server per zone. Three zones is why a cluster is three servers — lose a zone and two of three still holds a majority.



1 · NATS INSTANCE

One server process. It holds data and answers clients. Here, one per availability zone.

2 · CLUSTER

Three or more instances that **vote together**, inside one region. This is the unit of availability, and the only thing three servers buys you.

3 · GATEWAY

Joins two clusters and routes messages between them. It does **not** copy data — and it fuses both sides into one JetStream. See slide 06.

4 · LEAF NODE

A server that borrows a hub over one outbound link. It may run its own JetStream, which is the only place `jetstream.domain` belongs. See slide 07.

READ THE DOTTED BAND

The account is drawn **across** both regions, not inside one. Region and cell are places. Tenant and account are not, and a drawing that nests them inside a region is the mistake ADR-056 was written to stop.

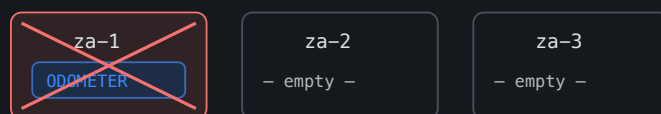
03

THE THREE-INSTANCE CLUSTER

Three servers is not three copies

A cluster is three instances because RAFT needs a majority to elect a leader, and the majority of three is two — so one instance can be lost. That protects the *cluster*. It does not protect your *stream*. Replication is a separate, per-stream choice, and its default is **one**.

REPLICAS: 1 — DEFAULT



stop the one server holding it

stream is offline (10118)

DATA STUCK UNTIL THAT SERVER RETURNS

REPLICAS: 3 — ASKED FOR, PER STREAM



stop the leader

new leader in about one second

NOTHING LOST · EVERY WRITE NOW COSTS THREE SERVERS

THE TRAP

- › The default is **1**, and nothing warns you. A three-server cluster with an R1 stream survives nothing.
- › **Replicas never cross a region.** `--replicas 3` means three copies in **za** — not one in each region.
- › R3 is slower and uses three times the disk. Choose it **on purpose, per stream**.

WHAT `--REPLICAS 3` ACTUALLY COPIES

COPIED 3 TIMES, INSIDE ZA

- › **Stream messages** — the log itself.
- › **Stream config** — subjects, limits, retention.
- › **Consumer position and acks** — its *own* vote group, which inherits 3 from the stream.
- › **KV bucket values** — a KV bucket **is** a stream.
- › **Object store blobs** — also a stream.

HANDLED SOMEWHERE ELSE

- › The **fact** that the stream exists. That lives in the meta group, on every server in the system — not in the replica set.

NOT COPIED AT ALL

- › **Anything in au** . A gateway routes; it does not copy. That needs a mirror.
- › **Ephemeral consumers** — R1, in memory, and they die with their server.

RECOMMENDATION

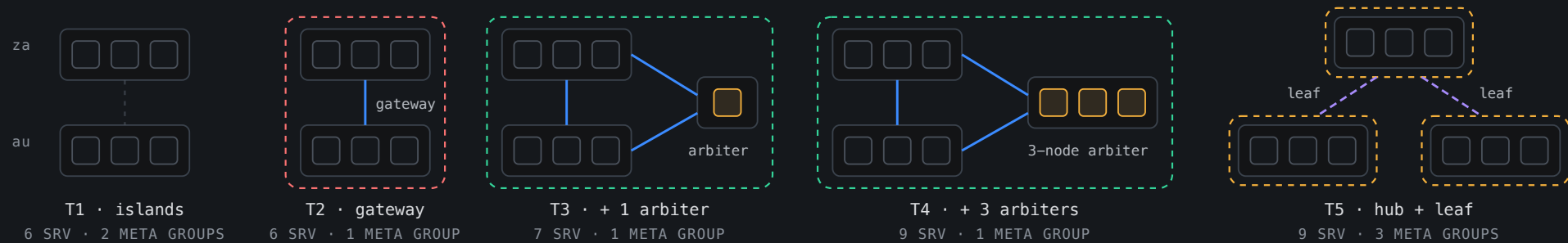
Start at three instances, one cluster. Set **Replicas: 3** explicitly on every stream and KV bucket that matters. Make it a reviewed default, not a habit.

04

FIVE WAYS TO ADD A SECOND REGION

Every shape is majority arithmetic

Demo 03 built seven shapes from nothing and measured each one. The single question that separates them is: **how many servers are in the JetStream meta group, and who still holds a majority when a region goes dark?** This slide only introduces them. The findings follow, one shape at a time.



Red ring = one shared fate. Green ring = one shared fate that still holds a majority when a region is lost. Amber rings = separate JetStream systems, each with its own vote.

THE TWO VARIANTS

VARIANT	WHAT IT ASKS
B · domain per cluster	T2, plus a different <code>jetstream.domain</code> on each side. Can a domain name split a supercluster?
E · export / import	T2, plus two accounts instead of one. Can a region share data deliberately, across the account wall?

WHAT EACH SHAPE IS

SHAPE	SERVICES	META GROUPS	WHAT IT IS
T1 islands	6	2	Two clusters, no link at all. Each region is its own NATS and its own JetStream.
T2 gateway	6	1	The same two clusters, joined by one gateway.
T3 + 1 arbiter	7	1	T2 plus a single-node third site that also votes.
T4 + 3 arbiters	9	1	T3 with a full three-node third site instead of one node.
T5 hub + leaf	9	3	A hub cluster, with a leaf cluster in each region. Each runs its own JetStream.

WHAT FOLLOWS

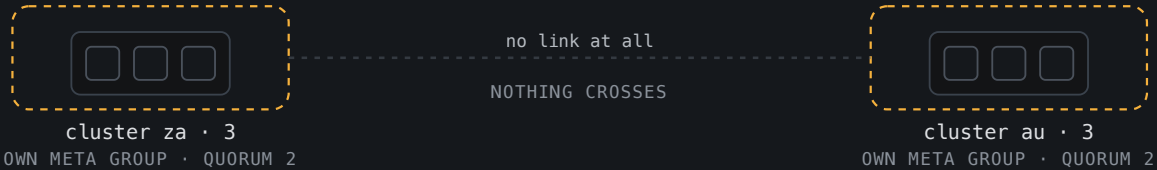
Five slides, one finding set per shape: 05 T1 · 06 T2 · 07 B · 08 T3 and T4 · 09 T5 and E. Every row on them is a check with an expected answer that matched.

05

T1 · NO LINK AT ALL · 6 SERVERS · 9 CHECKS, 9 PASSED

Two islands never freeze, and never agree

The baseline. Two clusters of three, with nothing joining them. Each region runs its own JetStream and its own vote. This is the only shape where a region going dark changes nothing at all for the other one — and the only shape that shares nothing whatsoever.



WHAT IT GIVES YOU

- › **Two separate meta groups**, of three servers each. A majority of three is two, so each region tolerates losing one server. [T1a](#) , [T1b](#) , [T1c](#)
- › **ZA goes dark and AU does not notice**. AU keeps its meta leader, creates a brand-new 3-replica stream, and its meta group size is unchanged at 3. [T1g](#) , [T1h](#) , [T1i](#)
- › No WAN on any read or write. Nothing to fail between the regions, because there is nothing between them.

WHAT IT COSTS YOU

- › **Two streams can carry the same name and be unrelated**. AU asks for [ODOMETER](#) , gets it, and it is a second real stream. No error, because there is no shared namespace to collide in. [T1d](#) , [T1e](#)
- › **Nothing crosses**. One publish in ZA: ZA holds 1 message, AU holds 0. [T1f](#)
- › Every cross-region copy is an application you have to write, run and monitor yourself.

MEASURED — DEMO 03, T1			
CHECK	QUESTION	EXPECTED	MEASURED
T1a	Distinct JetStream meta groups across both regions	2	2
T1f	One publish in ZA — messages in ZA / AU	1 / 0	1 / 0
T1g	ZA dark — AU still has a meta leader	yes	yes

CHOOSE T1 WHEN

The two regions genuinely share no data, and you would rather write one explicit copier than accept a shared vote. It is the safest shape for availability and the worst for consistency — and those are the same fact.

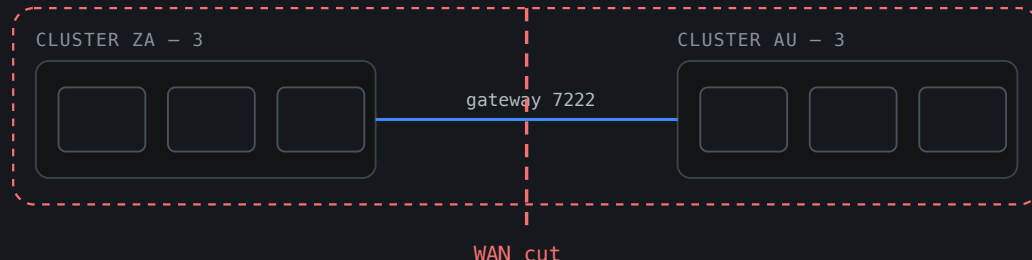
06

THE MOST EXPENSIVE THING ON THESE SLIDES

A gateway makes one JetStream, not two

This is the finding that changes a design. A gateway does not join two JetStream systems — it fuses them into one supercluster with **one meta group** and **one leader election**. The account wall still holds for data. It does not hold for availability.

ONE META GROUP · SIZE 6 · MAJORITY 4



leader: NONE

3 OF 6 — NOT A MAJORITY

leader: NONE

3 OF 6 — NOT A MAJORITY

WHAT A 3/3 SPLIT FREEZES

- > `stream add / rm / edit` → 10008 JetStream system temporarily unavailable
- > Every account at the same moment. Measured with 3 JetStream accounts live: **1 meta group**, not three.

WHAT KEEPS WORKING

- > Core NATS publish and subscribe.
- > Reads from a stream that already exists.
- > **Writes into a stream that already exists** — measured 10 → 15 messages, all acked.

ONE NAMESPACE MEANS ONE NAME

- > AU asks for a stream ZA already owns → 10058 stream name already in use
- > Overlapping subjects → 10065 subjects overlap
- > So inside one account a single publish can **never** be stored twice.

...AND ONE COPY, IN ONE PLACE

- > AU's client keeps working, because a stream is reachable from either side of a gateway.
- > But `stream info` from AU reports cluster **za**. AU holds nothing.
- > AU pays the WAN on every read and goes dark for that stream when ZA is lost.

UNEVEN REGIONS BEHAVE DIFFERENTLY

- > A **3 / 3** split freezes both sides.
- > A **3 / 2** split does not — the side holding 3 of 5 keeps its leader.
- > Which side survives is arithmetic, so design the count deliberately.

THE DECISION

If both regions must accept a JetStream **change** while the other is dark, a plain two-region gateway cannot do it. Add a vote outside both regions, or stop sharing a meta group.

07

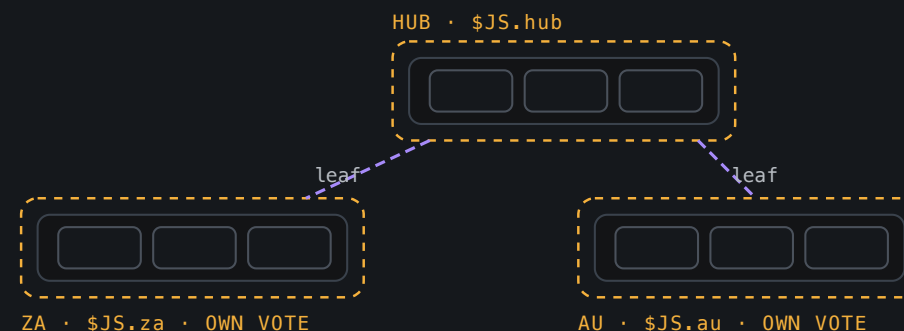
THE SETTING EVERYBODY REACHES FOR FIRST

A domain is a leaf tool. A gateway is not a leaf link.

`jetstream.domain` looks like the fix for the previous slide. It is not. We set it over a gateway, measured it, and it made things quietly worse. Over a **leaf** link the same setting is real and does exactly what the name promises.

A · DOMAIN OVER A GATEWAY — DOES NOT WORK

meta group size still 6 — domains did NOT split it
 one side never elects · placement → 10005 no suitable peers
 AND WHICH SIDE GOES BLIND IS A COIN TOSS — MEASURED BOTH WAYS

B · DOMAIN OVER A LEAF LINK — REAL**OVER A GATEWAY — MEASURED**

- › Meta group size **still 6**. A domain name cannot split a supercluster.
- › One cluster elects, the other reports `leader: null` forever.
- › Placing a stream on the blind cluster → **10005 no suitable peers for placement**
- › The verdict in the report is blunt: **does not work**.

OVER A LEAF LINK — MEASURED

- › **3 distinct meta groups**, sizes 3 / 3 / 3. Hub, ZA and AU each own their JetStream.
- › The whole hub can be stopped and both regions still elect, still create R3 streams, still store writes.
- › This is the only shape in the lab that survives the centre disappearing.

THE PRICE NOBODY MENTIONS

- › **Double capture is real here**. One publish in ZA was stored **1 and 1** — twice, in two systems, with no warning.
- › **Nothing replicates by itself**. Every cross-region copy is hand-written.
- › A mirror over a leaf needs `mirror.external.api` → `$JS.au.API`. Without it: **0 messages copied**, and the stream reports healthy.

THE RULE

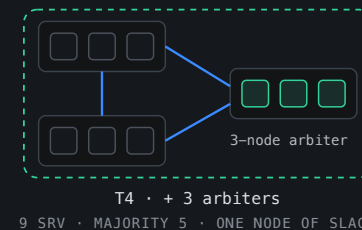
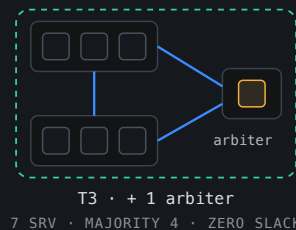
A JetStream domain is a leaf-link tool. Setting it over a gateway is not a smaller version of the same thing — it is a misconfiguration that reports success.

08

T3 · 7 SERVERS · 10 CHECKS — T4 · 9 SERVERS · 10 CHECKS — ALL PASSED

Buy a vote that sits outside both regions

T2 freezes because 3 of 6 is not a majority. Both fixes here are the same idea: put more voters in a third site, so a surviving region plus the third site still adds up. T3 adds one node. T4 adds three. The difference between them is **slack**, and it was measured, not assumed.



T3 — ONE ARBITER NODE · 7 SERVERS, MAJORITY 4

- One meta group across all three sites. [C1](#) , [C2](#)
- Either region can go dark and the survivors elect a leader** and still create a new 3-replica stream. [C3](#) — [C6](#)
- Zero slack.** 4 of 7 is exactly the majority. One more node after the region loss and it freezes. [C12](#)

T4 — A THREE-NODE ARBITER SITE · 9 SERVERS, MAJORITY 5

- Still one meta group, size 9. [F1](#) , [F2](#)
- A region dark leaves 6 of 9 — a leader is elected and writes continue. [F3](#) , [F4](#)
- One node of slack.** A region *plus* one arbiter node leaves 5 of 9, exactly the majority, and it still works. At 4 of 9 the leader is NONE. [F5](#) , [F6](#) , [F7](#) , [F11](#)
- Even with no majority, publishing into an **existing** stream still works. Only changes stop. [F8](#) , [F9](#)

THE ARBITER TRAP — IT WILL QUIETLY HOST YOUR DATA

- An explicit `--cluster arb` R1 stream **lands on the arbiter**, and a client dialling the arbiter's own port creates streams there too. [C8](#) , [C10](#)
- Asking for R3 on a one-node site fails with `10005` . [C9](#)
- Left alone, 8 unplaced R1 streams from a ZA client landed on the arbiter **zero** times — but that is the scheduler being kind, not a guarantee. [C7](#)
- So: keep clients off the arbiter's client port, or fence it with placement.** [C11](#)

ONE NUMBER WORTH REMEMBERING

After the majority was gone, `/jsz` went on naming a meta leader for **40 seconds**. [F7a](#)

And once a new leader was named, it accepted a change immediately — **0 seconds**. [F6a](#)

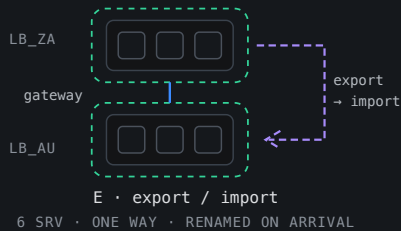
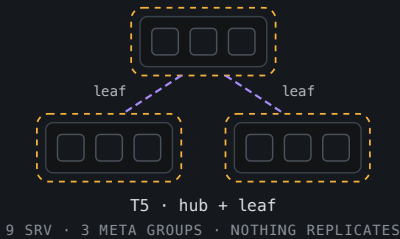
So the monitoring endpoint is late in one direction and early in the other. Neither edge can be trusted from a single reading.

09

T5 · 9 SERVERS · 14 CHECKS — E · 6 SERVERS · 8 CHECKS — ALL PASSED

Independence you can have, at the price of copying by hand

A leaf link is the other way out of T2. Each region keeps its own JetStream and its own vote, so the hub can vanish entirely and both regions carry on — but nothing then replicates by itself.



T5 — HUB AND LEAF · THREE SEPARATE JETSTREAM SYSTEMS

- Three meta groups, sizes 3 / 3 / 3. Each has its own quorum of two. [D1](#) , [D2](#)
- The whole hub can vanish. Both regions keep a leader, both create new 3-replica streams, and AU keeps storing its own publishes. [D10](#) — [D14](#)
- Both regions may own a stream called [ODOMETER](#) with no [10058](#) , because there is no shared namespace. [D3](#) , [D4](#)

T5 — WHAT IT COSTS

- Double capture is real here. ONE publish in ZA was stored in ZA and in AU — 1 / 1 — and two same-named shadow streams lived at once. [D5](#) , [D6](#)
- A mirror with no [external.api](#) copies nothing, and says nothing. 0 messages, no error, no warning, stream reports healthy. Add [\\$JS.au.API](#) and it copies. [D7](#) , [D7a](#) , [D8](#)
- Worse: a name-only mirror silently copied the local stream of the same name instead of the remote one. [D9](#) , [D9a](#)
- Nothing replicates by itself. Every cross-region copy is hand-written. [D15](#)

VARIANT E — EXPORT / IMPORT BETWEEN TWO ACCOUNTS, OVER THE SAME GATEWAY

CHECK	QUESTION	EXPECTED	MEASURED
E2	The importing account stored the publish too, across the gateway	1	1
E3	... and it arrived prefixed , so the raw-subject stream took none of it	0	0
E6 , E7	Importing account reads ZA's stream, then creates its OWN of that name	fails, then ok	fails, then ok

CHOOSE A LEAF LINK WHEN

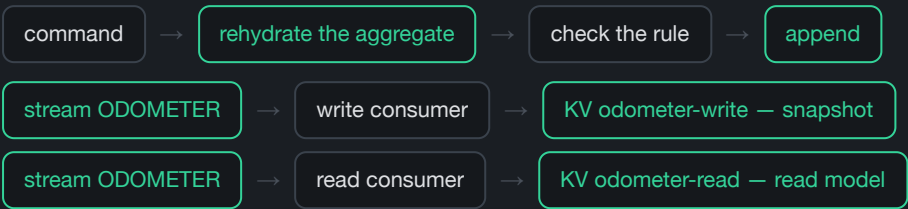
Regional independence matters more than one namespace — and budget for the copiers. Wherever you do share, share the way **E** does: across an account wall, one direction, renamed on arrival. Never by giving two streams the same name and hoping.

10 STREAMS, CONSUMERS, KV AND THE COST OF REPLAY

A KV bucket is a stream, and a consumer lives where its stream lives

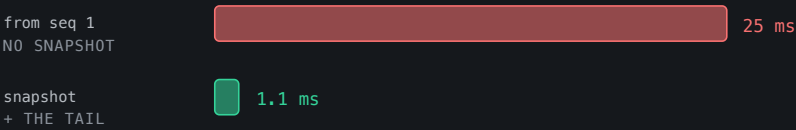
Placement is not a separate subject from clustering — it is the same subject seen from inside one cluster. And once state is rebuilt from a log, the length of that log becomes a latency number you have to manage.

THE CQRS SHAPE MEASURED IN DEMO 04



The log is the only source of truth. Delete both KV buckets and everything comes back. Delete the stream and nothing does.

REHYDRATION — 10 001 EVENTS ON ONE VEHICLE



ABOUT 23 TIMES FASTER — AND THE GAP GROWS WITH THE LOG

Replay is **linear** in the length of the log. The snapshot read is one KV get and does not care how long the log is. Take the log to 100k events and the left-hand bar grows tenfold while the right-hand one does not move.

PLACEMENT — THE RULES WE MEASURED

- › Everything lands **where the account's first request landed**, unless you name a cluster with `--cluster`.
- › A **KV bucket follows the same rule, because a bucket is a stream**.
- › A durable pull consumer created from AU for a ZA-held stream reported cluster **za**. AU held no part of it and paid the WAN on every pull.

TWO SETTINGS THAT FAIL SILENTLY

- › **LimitsPolicy, not InterestPolicy**, on any stream you intend to replay. InterestPolicy discards a message once every consumer has acked it, and the replay then returns nothing — with no error.
- › **The snapshot is always stale**. Rehydration must read the snapshot *and then replay the tail*. Measured mid-catch-up: 2 900 events of tail in 12 ms.

STORAGE — THE STANDING RULE

- › **A count is never shown without its bytes**. 100 000 000 events sounds reasonable right up to the moment you learn it is **7.6 GB**.
- › R3 triples that figure, per stream, per region.
- › Seeding one million events into the bench fixture takes about **2.2 seconds** — the log grows faster than anyone plans for.

RECOMMENDATION

Treat snapshotting as a **requirement**, not an optimisation, on any aggregate that will take more than a few thousand events. And state a retention policy on day one, in bytes.

11

FINDINGS THIS LAB ADDED

Five failures that report success

These are not in the documentation. Each one was found by a script that expected a specific answer and got a different one. Every one of them looks healthy from the outside — which is exactly why they are on a slide.

01 · THE MONITORING ENDPOINT LIES WHILE IT IS STALE

After the majority of a supercluster was frozen, `/jsz` kept naming a meta leader for **roughly a minute**. Read it too early and a dead supercluster looks perfectly healthy.

In that window the client does not get a clean refusal either — **it hangs and times out**. The clean `10008` only appears once the old leader has aged out.

02 · A NEW LEADER IS NAMED BEFORE IT CAN TAKE A CHANGE

After an election, `/jsz` names the new leader **seconds before** that leader will accept a create. Ask immediately and the call is refused — on a supercluster that is in fact healthy.

So **neither edge of a failover can be trusted from a single reading**. Health checks must retry and record how long they waited, not sleep a magic number.

03 · A NAME-ONLY MIRROR CAN COPY THE WRONG STREAM

A mirror with no `external.api` does not fail when the name it wants only exists in the other region — **it just sits at zero**. Worse: if a stream of the same name also exists locally, the mirror copies *that* one, reports healthy, and looks like it is working.

Proved by giving the two streams different message counts. No error, no warning, wrong data.

04 · WHICH REGION GOES BLIND IS A COIN TOSS

Two runs of the same configuration, minutes apart, nothing changed — and the blind side swapped.

A team that tests this shape once will write down the wrong region's name. Our check deliberately refuses to name a side.

05 · A TEST THAT DOES NOT TEST READS EXACTLY LIKE A RESULT

Not a NATS fact — a rig fact, and the most important one. An early version of the harness recorded the wrong process id, so `kill -STOP` hit nothing and the “frozen” region stayed fully alive. **The lab reported healthy behaviour and it looked like a finding**. Every freeze in these scripts now proves itself first, by checking the monitor port has really gone dark.

TWO NUMBERS WE RETRACTED

- › A cross-region “double capture” of **50 km** was a replay bug — a durable consumer was being deleted on every run.
- › A rehydration speed-up of **600x** was **99% consumer bookkeeping**, not log reading. The real figure is 23x.
- › **A lab whose numbers only ever improve is not measuring.**

OPERATIONAL CONSEQUENCE

Do not build alerting, failover automation or a runbook on a single read of `/jsz`. Every one of these failures passes a naive health check.

12 WHAT THIS LETS SOMEBODY DECIDE

Pick the shape from the requirement, not the diagram

Every row below is a requirement somebody will actually state, and the shape the measurements say it selects. The mistakes happen when a shape is chosen first and a requirement is fitted to it afterwards.

IF THE REQUIREMENT IS...	...THE SHAPE IS	...AND THIS IS WHAT IT COSTS
One tenant must not see another tenant's data	An account. Not a cluster, not a domain.	Nothing. It is the one wall NATS enforces itself, and it holds across a gateway.
One name, one copy, no duplicates possible	Gateway (T2)	Shared availability. A 3/3 split freezes every account at once.
Both regions must accept a JetStream change while the other is dark	A vote outside both regions — T3 (7) or T4 (9)	A third site. T3 has zero slack afterwards; T4 has exactly one node.
A region must keep running when the centre is gone	Leaf links + a domain per region (T5)	Double capture, and every cross-region copy hand-written as a mirror.
Two accounts must share one subject on purpose	Export / import	One-directional, read-only, renamed on arrival. The only safe sharing measured.
Data must stay in-country (residency)	A regional cell, gateway-joined, under the one global operator	Placement discipline: the tenant is pinned, no stream mirrors or sources leave the cell.
Access and control must stay in-country (sovereignty)	A sovereign cell — own operator, own resolver, no gateway	A second provisioning act, a second key chain to rotate, a second resolver to run.
A depot or yard must survive losing its link	A leaf node	It is never the answer to a cross-border requirement — it puts the trust root outside the border.

WHERE TO START — AND STAY

- **One cluster of three.** It survives one loss, and it is the whole of what most of the platform needs.
- **Replicas: 3** set explicitly on every stream and KV bucket that matters.
- Accounts as the tenant wall from day one. Retrofitting that boundary later is the expensive move.

WHAT WE HAVE NOT MEASURED YET

- The **latency** cost of a cross-region read. Placement is measured; latency is not.
- **Mirror lag and bandwidth** in either shape.
- A hub that holds data of its own, rather than only relaying.
- Leaf reconnect with more than two regions.

THE NEXT QUESTION

Every shape here is available today and we can pick one on the evidence. What we cannot yet price is **the wire**: what a cross-border byte costs in latency and in bandwidth.

That is the next demo, and it is the one that decides whether a gateway between regions is affordable at all.

13 WHERE EVERY NUMBER CAME FROM

A figure with no provenance becomes a stale constant

Each number on these slides, the day it was measured, and what it ran on. Nothing here was taken from documentation or from memory.

THE RUNS

SOURCE	MEASURED	ON	WHAT IT PRODUCED
Demo 03 — <code>lab/run-all.sh</code>	2026-09-17 20:28 SAST	nats-server 2.14.6 · Darwin 25.4.0 arm64	73 checks passed, 0 failed, 19 notes, across 7 topologies. Meta group sizes, 10008 / 10058 / 10005 refusals, mirror behaviour, all five findings.
Demo 02 — multi-region lab scripts	2026-09-09 & 2026-09-11	nats-server 2.14 in Docker, one host	Replicas 1 vs 3, the account wall across a gateway, WAN cut = change freeze only (10 → 15 messages acked), subject overlap 10065.
Demo 04 — <code>cqrs rehydrate</code>	2026-09-16	nats-server 2.14.3 in Docker, on a laptop	10 001 events on one vehicle, best of three runs each way: 25 ms without a snapshot, 1.1 ms with one.
ADR-054 / ADR-056	Accepted 2026-09-07 / 2026-09-18	V3 platform architecture record	The five axes, the tenant-wall rule, and the three cross-border shapes (leaf node, regional cell, sovereign cell).

RETRACTIONS — BOTH CORRECTIONS ARE ON THE RECORD

- › “25 / 25 / 50 km cross-region double capture” is withdrawn. The 25 came from a durable pull consumer being deleted on every projector run, so each run replayed the whole stream. Four runs over a stream holding **one** message read 12.5, 25, 37.5, 50. The 50 came from adding one region’s number to the other while both read the same bucket. Behind a gateway there is nothing to capture twice — **double capture is real in the hub-and-leaf shape instead**.
- › “8.2 s, about 600x” is withdrawn. The replay used an ordered consumer’s `Next()`, which resets the consumer on every call — the demo was creating one consumer per event. About 99% of that time was consumer bookkeeping. The corrected figure is **25 ms vs 1.1 ms, about 23x**. The lesson survived the correction; the number now measures the lesson instead of a bug.

HOW TO REPRODUCE ANY OF IT

- › Demo 03, all seven topologies: `demos/03-multi-cluster-and-accounts/lab/run-all.sh` — builds every shape from nothing and regenerates `REPORT.md` and `REPORT.html`. Both are generated; neither is hand-edited.
- › Demo 02, replicas and the account wall: `demos/02-multi-region/lab/01-the-wall.sh`, `02-replicas.sh`, `03-odometer.sh`.
- › Demo 04, rehydration: `cqrs seed -vehicle V1 -n 10000`, then `cqrs rehydrate -snapshot=false`; wait about 20 seconds for the asynchronous snapshotter, then `cqrs rehydrate -snapshot=true`.
- › A caution that applies to all three: never fake a WAN cut with `docker network disconnect` — it drops published host ports, and an early run wrongly reported that both regions had died.